

Q4. What is the *main goal* of asymptotic complexity analysis? [mark-1]
ANS: To understand how an algorithm's resource usage grows with input size

Q5. Match the complexity notation with their growth type. [mark 3= 0.5x6]

Complexity Notation	Growth Type
$O(1)$	Constant
$O(N)$	Linear
$O(N \log N)$	Log-linear
$O(N^2)$	Quadratic
$O(N^3)$	Cubic
$O(N!)$	Factorial

Q4.

Q4A. [lowest time] $O(\log n) < O(n) < O(n^2)$ [highest time]

Q4B. [lowest time] $O(n) < O(n \log n) < O(n^2)$ [highest time]

Q48C. [lowest time] $O(n^2) < O(2^n) < O(n!)$ [highest time]

Q4D. [lowest time] $O(1) < O(\log n) < O(n \log n)$ [highest time]

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Q1A. Given a 4D array of size $2 \times 3 \times 4 \times 5$, find the multidimensional index for the linear index 87. What is the length of the 1D array?

$w \times (N \times O \times P) + x \times (O \times P) + y \times P + z = \text{Linear Index}$, Where w, x, y, z represent the 4D indices, and $M=2, N=3, O=4, P=5$

The length of the array in 1D = $M \times N \times O \times P = 2 \times 3 \times 4 \times 5 = 120$

So the formula becomes:

$$w \times (3 \times 4 \times 5) + x \times (4 \times 5) + y \times 5 + z = 87$$

$$w \times 60 + x \times 20 + y \times 5 + z = 87$$

$$w = 87 // (3 \times 4 \times 5) = 87 // 60 = 1$$

$$\text{Remainder} = 87 \% 60 = 27$$

$$x = 27 // (4 \times 5) = 27 // 20 = 1$$

$$\text{Remainder} = 27 \% 20 = 7$$

$$y = 7 // 5 = 1$$

$$\text{Remainder} = 7 \% 5 = 2$$

$$z = \text{Remainder from previous step} = 2$$

$$\text{4D Index for Linear Index 87: } [1][1][1][2]$$

Q1B. Given a 4D array of size $4 \times 2 \times 3 \times 6$, find the multidimensional index for the linear index 101. What is the length of the 1D array?

$$w \times (N \times O \times P) + x \times (O \times P) + y \times P + z = \text{Linear Index, Where } w, x, y, z \text{ represent the 4D indices, and } M=4, N = 2, O = 3, P = 6$$

$$\text{The length of the array in 1D} = M \times N \times O \times P = 4 \times 2 \times 3 \times 6 = 144$$

So the formula becomes:

$$w \times (2 \times 3 \times 6) + x \times (3 \times 6) + y \times 6 + z = 101$$

$$w \times 36 + x \times 18 + y \times 6 + z = 101$$

$$w = 101 // (2 \times 3 \times 6) = 101 // 36 = 2$$

$$\text{Remainder} = 101 \% 36 = 29$$

$$x = 29 // (3 \times 6) = 29 // 18 = 1$$

$$\text{Remainder} = 29 \% 18 = 11$$

$$y = 11 // 6 = 1$$

$$\text{Remainder} = 11 \% 6 = 5$$

$$z = \text{Remainder from previous step} = 5$$

$$\text{4D Index for Linear Index 101: } [2][1][1][5]$$

Q1C. Given a 4D array of size $3 \times 5 \times 2 \times 4$, find the multidimensional index for the linear index 73. What is the length of the 1D array?

$$w \times (N \times O \times P) + x \times (O \times P) + y \times P + z = \text{Linear Index, Where } w, x, y, z \text{ represent the 4D indices, and } M=3, N = 5, O = 2, P = 4$$

$$\text{The length of the array in 1D} = M \times N \times O \times P = 3 \times 5 \times 2 \times 4 = 120$$

So the formula becomes:

$$w \times (5 \times 2 \times 4) + x \times (2 \times 4) + y \times 4 + z = 73$$

$$w \times 40 + x \times 8 + y \times 4 + z = 73$$

$$w = 73 // (5 \times 2 \times 4) = 73 // 40 = 1$$

$$\text{Remainder} = 73 \% 40 = 33$$

$$x = 33 // (2 \times 4) = 33 // 8 = 4$$

$$\text{Remainder} = 33 \% 8 = 1$$

$$y = 1 // 4 = 0$$

$$\text{Remainder} = 1 \% 4 = 1$$

$$z = \text{Remainder from previous step} = 1$$

4D Index for Linear Index 73: [1][4][0][1]

Q1D. Given a 4D array of size $2 \times 4 \times 3 \times 7$, find the multidimensional index for the linear index 154. What is the length of the 1D array?

$$w \times (N \times O \times P) + x \times (O \times P) + y \times P + z = \text{Linear Index, Where } w, x, y, z \text{ represent the 4D indices, and } M=2, N=4, O=3, P=7$$

$$\text{The length of the array in 1D} = M \times N \times O \times P = 2 \times 4 \times 3 \times 7 = 168$$

So the formula becomes:

$$w \times (4 \times 3 \times 7) + x \times (3 \times 7) + y \times 7 + z = 154$$

$$w \times 84 + x \times 21 + y \times 7 + z = 154$$

$$w = 154 // (4 \times 3 \times 7) = 154 // 84 = 1$$

$$\text{Remainder} = 154 \% 84 = 70$$

$$x = 70 // (3 \times 7) = 70 // 21 = 3$$

$$\text{Remainder} = 70 \% 21 = 7$$

$$y = 7 // 7 = 1$$

$$\text{Remainder} = 7 \% 7 = 0$$

$$z = \text{Remainder from previous step} = 0$$

4D Index for Linear Index 154: [1][3][1][0]

Q2A. Suppose a multidimensional 5D array has the dimensions of $3 \times 4 \times 2 \times 5 \times 6$. For a value stored at index [2][1][0][3][4] in the 5D array, what will be its index in the linear array?

Given,

Dimension of the 5D array: $3 \times 4 \times 2 \times 5 \times 6$

Since the array has dimensions of $3 \times 4 \times 2 \times 5 \times 6$, then: $L=3, M=4, N=2, O=5, P=6$

Index of a value in 5D array is: [2][1][0][3][4]

Let's assume: $(w, x, y, z, q) = (2, 1, 0, 3, 4) \rightarrow$ So, $w=2, x=1, y=0, z=3, q=4$

Find: We need to find the index of the same value in the 1D array (5D to 1D).

$$\text{linear_index} = w \times (M \times N \times O \times P) + x \times (N \times O \times P) + y \times (O \times P) + z \times P + q$$

Substituting values:

$$\text{linear_index} = 2 \times (4 \times 2 \times 5 \times 6) + 1 \times (2 \times 5 \times 6) + 0 \times (5 \times 6) + 3 \times 6 + 4$$

$$\text{linear_index} = 2 \times 240 + 1 \times 60 + 0 + 18 + 4$$

$$\text{linear_index} = 480 + 60 + 0 + 18 + 4 = 562$$

Q2B. Suppose a multidimensional 5D array has the dimensions of $2 \times 3 \times 4 \times 2 \times 5$. For a value stored at index $[1][2][3][1][4]$ in the 5D array, what will be its index in the linear array?

Given,

Dimension of the 5D array: $2 \times 3 \times 4 \times 2 \times 5$

Since the array has dimensions of $2 \times 3 \times 4 \times 2 \times 5$, then: $L=2$, $M = 3$, $N = 4$, $O = 2$, $P = 5$

Index of a value in 5D array is: $[1][2][3][1][4]$

Let's assume: $(w, x, y, z, q) = (1, 2, 3, 1, 4) \rightarrow$ So, $w = 1$, $x = 2$, $y = 3$, $z = 1$, $q = 4$

Find: We need to find the index of the same value in the 1D array (5D to 1D).

$$\text{linear_index} = w \times (M \times N \times O \times P) + x \times (N \times O \times P) + y \times (O \times P) + z \times P + q$$

Substituting values:

$$\text{linear_index} = 1 \times (3 \times 4 \times 2 \times 5) + 2 \times (4 \times 2 \times 5) + 3 \times (2 \times 5) + 1 \times 5 + 4$$

$$\text{linear_index} = 1 \times 120 + 2 \times 40 + 3 \times 10 + 5 + 4$$

$$\text{linear_index} = 120 + 80 + 30 + 5 + 4 = 239$$

Q2C. Suppose a multidimensional 5D array has the dimensions of $4 \times 2 \times 3 \times 5 \times 2$. For a value stored at index $[3][1][2][4][1]$ in the 5D array, what will be its index in the linear array?

Given,

Dimension of the 5D array: $4 \times 2 \times 3 \times 5 \times 2$

Since the array has dimensions of $4 \times 2 \times 3 \times 5 \times 2$, then: $L=4$, $M = 2$, $N = 3$, $O = 5$, $P = 2$

Index of a value in 5D array is: $[3][1][2][4][1]$

Let's assume: $(w, x, y, z, q) = (3, 1, 2, 4, 1) \rightarrow$ So, $w = 3$, $x = 1$, $y = 2$, $z = 4$, $q = 1$

Find: We need to find the index of the same value in the 1D array (5D to 1D).

$$\text{linear_index} = w \times (M \times N \times O \times P) + x \times (N \times O \times P) + y \times (O \times P) + z \times P + q$$

Substituting values:

$$\text{linear_index} = 3 \times (2 \times 3 \times 5 \times 2) + 1 \times (3 \times 5 \times 2) + 2 \times (5 \times 2) + 4 \times 2 + 1$$

$$\text{linear_index} = 3 \times 60 + 1 \times 30 + 2 \times 10 + 8 + 1$$

$$\text{linear_index} = 180 + 30 + 20 + 8 + 1 = 239$$

Q2D. Suppose a multidimensional 5D array has the dimensions of 2×5×3×4×7. For a value stored at index [1][3][2][1][5] in the 5D array, what will be its index in the linear array?

Given,

Dimension of the 5D array: 2×5×3×4×7

Since the array has dimensions of 2×5×3×4×7, then: L=2, M = 5, N = 3, O = 4, P = 7

Index of a value in 5D array is: [1][3][2][1][5]

Let’s assume: (w, x, y, z, q) = (1, 3, 2, 1, 5) → So, w = 1, x = 3, y = 2, z = 1, q = 5

Find: We need to find the index of the same value in the 1D array (5D to 1D).

$$\text{linear_index} = w \times (M \times N \times O \times P) + x \times (N \times O \times P) + y \times (O \times P) + z \times P + q$$

Substituting values:

$$\text{linear_index} = 1 \times (5 \times 3 \times 4 \times 7) + 3 \times (3 \times 4 \times 7) + 2 \times (4 \times 7) + 1 \times 7 + 5$$

$$\text{linear_index} = 1 \times 420 + 3 \times 84 + 2 \times 28 + 7 + 5$$

$$\text{linear_index} = 420 + 252 + 56 + 7 + 5 = 740$$

Java	Python
<pre>// Solution without size public static int product_first_n(Node head, int n) { if (head == null n <= 0) { return 0; } int product = 1; int count = 0; Node current_node = head; while (current_node != null && count < n) { product *= current_node.elem; count++; current_node = current_node.next; } if (count < n) { System.out.println("Invalid n: n is greater than size."); return 0; } return product; }</pre>	<pre>#Solution without size def product_first_n(head, n): if head is None or n <= 0: return 0 product = 1 count = 0 current_node = head while current_node is not None and count < n: product *= current_node.elem count += 1 current_node = current_node.next if count < n: print("Invalid n: n is greater than size.") return 0 return product</pre>

#Solution with size

```
public static int getSize(Node head) {
    if (head == null) {
        return 0;
    }

    Node current_node = head;
    int count = 0;

    while (current_node != null) {
        current_node = current_node.next;
        count++;
    }

    return count;
}

public static int product_first_n(Node head, int n) {
    int size = getSize(head);

    if (head == null || n <= 0) {
        return 0;
    }

    if (n > size) {
        System.out.println("Invalid n: n is greater than size.");
        return 0;
    }

    int product = 1;
    int count = 0;
    Node current_node = head;

    while (count < n) {
        product *= current_node.elem;
        count++;
        current_node = current_node.next;
    }

    return product;
}
```

// Solution without size

```
public static int sum_first_n(Node head, int n) {
    if (head == null || n <= 0) {
        return 0;
    }

    int sum = 0;
    int count = 0;
    Node current_node = head;

    while (current_node != null && count < n) {
        sum += current_node.elem;
        count++;
        current_node = current_node.next;
    }

    if (count < n) {
        System.out.println("Invalid n: n is greater than size");
        return 0;
    }
}
```

#Solution with size

```
def getSize(head):
    if head == None:
        return 0

    current_node = head
    count = 0

    while current_node != None:
        current_node = current_node.next
        count += 1

    return count

def product_first_n(head, n):
    size = getSize(head)

    if head == None or n <= 0:
        return 0

    if n > size:
        print("Invalid n: n is greater than size.")
        return 0

    product = 1
    count = 0
    current_node = head

    while count < n:
        product *= current_node.elem
        count += 1
        current_node = current_node.next

    return product
```

// Solution without size

```
def sum_first_n(head, n):
    if head is None or n <= 0:
        return 0

    total = 0
    count = 0
    current_node = head

    while current_node is not None and count < n:
        total += current_node.elem
        count += 1
        current_node = current_node.next

    if count < n:
        print("Invalid n: n is greater than size.")
        return 0

    return total
```

<pre> return sum; } </pre>	
#Solution with size <pre> public static int getSize(Node head) { if (head == null) { return 0; } Node current_node = head; int count = 0; while (current_node != null) { current_node = current_node.next; count++; } return count; } public static int sum_first_n(Node head, int n) { int size = getSize(head); if (head == null n <= 0) { return 0; } if (n > size) { System.out.println("Invalid n: n is greater than size."); return 0; } int sum = 0; int count = 0; Node current_node = head; while (count < n) { sum += current_node.elem; count++; current_node = current_node.next; } return sum; } </pre>	#Solution with size <pre> def getSize(head): if head == None: return 0 current_node = head count = 0 while current_node != None: current_node = current_node.next count += 1 return count def sum_first_n(head, n): size = getSize(head) if head == None or n <= 0: return 0 if n > size: print("Invalid n: n is greater than size.") return 0 total = 0 count = 0 current_node = head while count < n: total += current_node.elem count += 1 current_node = current_node.next return total </pre>
<pre> public static int count_greater_than(Node head, int x) { if (head == null) { return 0; } int count = 0; Node current_node = head; while (current_node != null) { if (current_node.elem > x) { count++; } current_node = current_node.next; } return count; } </pre>	<pre> def count_greater_than(head, x): if head is None: return 0 count = 0 current_node = head while current_node is not None: if current_node.elem > x: count += 1 current_node = current_node.next return count </pre>

```
public static int sum_even_nodes(Node head) {  
    if (head == null) {  
        return 0;  
    }  
  
    int sum = 0;  
    Node current_node = head;  
  
    while (current_node != null) {  
        if (current_node.elem % 2 == 0) {  
            sum += current_node.elem;  
        }  
        current_node = current_node.next;  
    }  
  
    return sum;  
}
```

```
def sum_even_nodes(head):  
    if head is None:  
        return 0  
  
    total = 0  
    current_node = head  
  
    while current_node is not None:  
        if current_node.elem % 2 == 0:  
            total += current_node.elem  
            current_node = current_node.next  
  
    return total
```